

Paralelismo ao Nível de Instrução (ILP)

Dependências de dados e níveis de otimização em C

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

2026

1 Enunciado

Implementar três laços em C para investigar ILP: inicializar um vetor com um cálculo simples, somar seus elementos com dependência entre iterações e repetir a soma com múltiplos acumuladores. Comparar versões compiladas com -O0, -O2 e -O3, analisando o efeito do estilo do código e das dependências no desempenho.

2 Fundamentação teórica

ILP é a capacidade de uma CPU executar simultaneamente instruções independentes de um mesmo fluxo. No laço dependente, cada soma precisa do resultado anterior: $s_{i+1} = s_i + v[i + 1]$. Essa cadeia limita o número de somas que podem ser iniciadas em paralelo. Com quatro acumuladores, as iterações são distribuídas entre quatro cadeias independentes; o processador e o compilador podem explorar mais ILP. O laço de inicialização não possui dependência entre escritas diferentes, o que facilita a vetorização.

3 Experimento

O vetor contém $N = 100.000.000$ inteiros e é inicializado por $v[i] = 3*i + 1$. Depois, os dois laços de soma percorrem o mesmo vetor. Os tempos foram medidos apenas no corpo de cada laço com `clock_gettime(CLOCK_MONOTONIC)`. O programa também verifica que as duas somas produzem o mesmo resultado. Comandos de compilação:

```
1 gcc -std=c11 -Wall -Wextra -O0 tarefa03.c -o tarefa03_00
2 gcc -std=c11 -Wall -Wextra -O2 tarefa03.c -o tarefa03_02
3 gcc -std=c11 -Wall -Wextra -O3 tarefa03.c -o tarefa03_03
```

4 Código-fonte

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6
```

```

7  #define DEFAULT_N 100000000L
8
9  static double agora(void) {
10     struct timespec t;
11     clock_gettime(CLOCK_MONOTONIC, &t);
12     return t.tv_sec + t.tv_nsec / 1e9;
13 }
14
15 int main(int argc, char **argv) {
16     long n = argc > 1 ? atol(argv[1]) : DEFAULT_N;
17     if (n <= 0) {
18         fprintf(stderr, "Uso: %s [numero_de_elementos]\n", argv[0]);
19         return 1;
20     }
21
22     int *v = malloc((size_t)n * sizeof(*v));
23     if (!v) {
24         perror("malloc");
25         return 1;
26     }
27
28     double inicio = agora();
29     for (long i = 0; i < n; ++i)
30         v[i] = (int)(3 * i + 1);
31     double tempo_inicializacao = agora() - inicio;
32
33     inicio = agora();
34     long long soma_dependente = 0;
35     for (long i = 0; i < n; ++i)
36         soma_dependente += v[i];
37     double tempo_dependente = agora() - inicio;
38
39     inicio = agora();
40     long long s0 = 0, s1 = 0, s2 = 0, s3 = 0;
41     long i = 0;
42     for (; i + 3 < n; i += 4) {
43         s0 += v[i];
44         s1 += v[i + 1];
45         s2 += v[i + 2];
46         s3 += v[i + 3];
47     }
48     for (; i < n; ++i)
49         s0 += v[i];
50     long long soma_multiplos = s0 + s1 + s2 + s3;
51     double tempo_multiplos = agora() - inicio;
52
53     printf("N=%ld inicializacao=%.9f dependente=%.9f multiplos=%.9f "
54           "somadas=%lld,%lld\n", n, tempo_inicializacao,
55           tempo_dependente, tempo_multiplos, soma_dependente,

```

```

56     soma_multiplos);
57     free(v);
58     return soma_dependente != soma_multiplos;
59 }

```

Listing 1: tarefa03.c

5 Resultados

Table 1: Tempo em segundos para $N = 100.000.000$.

Otimização	Inicialização	Soma dependente	Múltiplos acumuladores
O0	0,300583	0,151139	0,058475
O2	0,102487	0,028251	0,026156
O3	0,109349	0,026172	0,030444

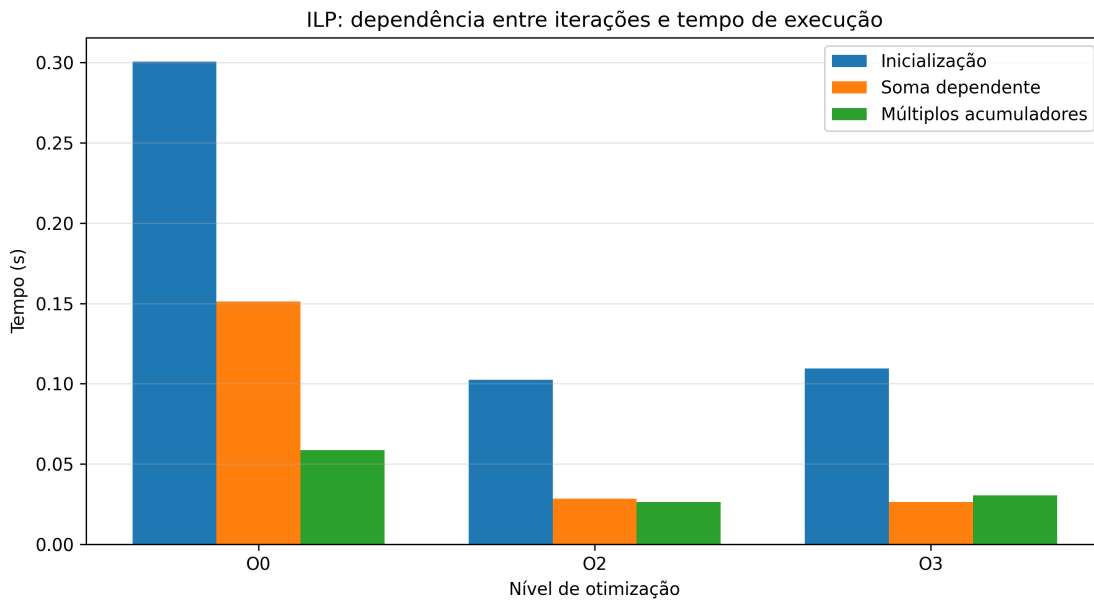


Figure 1: Tempo de execução dos três laços por nível de otimização ($N = 100.000.000$).

Em -O0, a soma com múltiplos acumuladores levou 0,058 s contra 0,151 s da soma dependente, 2,6× mais rápida, porque expõe quatro cadeias de cálculo independentes. Em -O2 e -O3, o compilador já transforma ou vetoriza partes dos laços, e a vantagem explícita dos acumuladores diminui. O3 não foi mais rápido que O2 no terceiro laço nesta execução (0,030 s contra 0,026 s). A diferença é pequena e pode resultar de ruído, frequência da CPU ou escolhas distintas de código gerado. Uma inspeção do assembly (`gcc -S`) permitiria verificar se O3 escolheu uma estratégia de vetorização diferente de O2, mas essa análise fica como trabalho futuro. A inicialização caiu de 0,301 s em O0 para 0,109 s em O3.

6 Discussão

As três versões realizam $O(N)$ operações, mas não apresentam o mesmo custo constante. A dependência do acumulador único forma um caminho crítico de N somas encadeadas. Os acumuladores múltiplos quebram esse caminho e permitem sobreposição de somas. Em troca, exigem uma redução final dos quatro parciais. Com -O2 ou -O3, o compilador aplica desenrolamento e vetorização sem alteração manual do código, e a diferença entre os estilos diminui.

O experimento é sequencial: ILP ocorre dentro de um núcleo e não deve ser confundido com paralelismo entre threads ou núcleos. Os tempos são de uma execução e podem variar com carga do sistema, frequência e estado das caches. Repetições e mediana reduziram esse ruído.

7 Conclusão

O experimento mostrou que dependências entre iterações limitam o ILP. Sem otimização, a soma com quatro acumuladores foi $2,6\times$ mais rápida. Com O2 e O3, o compilador explorou boa parte desse paralelismo por conta própria. O padrão de dependências afeta o tempo de execução de forma comparável à complexidade assintótica.

References

- [1] J. L. Hennessy e D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6ª ed. Morgan Kaufmann, 2019.
- [2] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2024.